

Uczenie Maszynowe

08.06.2026

Zmodyfikowana wersja algorytmu AQ z wykorzystaniem zbioru walidacyjnego

Autor:

Adrian Garbowski, Wiktor Sosnowski

Jednostka:

Wydział Elektroniki i Technik Informatycznych Politechniki Warszawskiej (EiT PW)



Spis treści

1	Opis projektu	3
2	Podstawy teoretyczne	3
2.1	Algorytm AQ	3
2.2	Zmodyfikowana wersja ze zbiorem walidacyjnym	4
2.3	Metryki oceny	5
3	Implementacja	5
4	Zbiory danych	6
4.1	Car Evaluation	6
4.2	Mushrooms	6
4.3	Nursery	6
4.4	Podział danych	7
5	Eksperymenty i wyniki	7
5.1	Eksperyment 1 – Porównanie wariantów (wyniki główne)	7
5.2	Eksperyment 2 – Wpływ szerokości wiązki	9
5.3	Eksperyment 3 – Wpływ rozmiaru zbioru walidacyjnego	10
6	Wnioski i podsumowanie	11

1 Opis projektu

Tematem projektu jest implementacja zmodyfikowanej wersji algorytmu indukcji reguł AQ, w której do oceny kompleksu dodawanego w kolejnych iteracjach wykorzystywany jest osobny zbiór walidacyjny.

Celem projektu jest porównanie klasycznej wersji algorytmu AQ z wersją zmodyfikowaną i sprawdzenie, czy ocenianie kompleksów na osobnym zbiorze walidacyjnym może ograniczyć przeuczenie oraz poprawić wyniki na nowych danych, przy czym efekt ten może zależeć od charakterystyki zbioru. Projekt zakłada implementację obu wersji algorytmu, przeprowadzenie eksperymentów na trzech zbiorach danych oraz analizę uzyskanych wyników.

2 Podstawy teoretyczne

2.1 Algorytm AQ

Algorytm AQ (Algorithm Quasi-optimal) należy do klasy algorytmów indukcji reguł i służy do budowy klasyfikatorów w postaci zestawu reguł logicznych [1, 2]. Reguły mają postać:

$$\text{JEŚLI } c_1 \wedge c_2 \wedge \dots \wedge c_k \text{ TO } \textit{klasa} = K$$

gdzie każdy warunek c_i ogranicza wartość jednego atrybutu do wybranego podzbioru dopuszczalnych wartości. Zbiór takich warunków nazywany jest **kompleksem**. Kompleks *pokrywa* dany przykład, jeśli wszystkie jego warunki są spełnione.

Klasyczna wersja algorytmu

Dla każdej klasy docelowej K algorytm wykonuje następujące kroki:

1. Wybierz losowy przykład pozytywny (należący do klasy K) jako ziarno (*seed*).
2. Generuj **gwiazdę** – zbiór kompleksów pokrywających ziarno i wykluczających przykłady negatywne. Kandydaci są specjalizowani poprzez dodawanie kolejnych warunków, tak aby żaden z kompleksów nie pokrywał żadnego przykładu negatywnego.
3. Spośród kandydatów wybierz **najlepszy kompleks** według miary jakości (w implementacji: F_1 wyważone między precyzją a pokryciem).
4. Utwórz regułę na podstawie wybranego kompleksu i usuń ze zbioru niepokrytych wszystkie przykłady pozytywne pokryte tą regułą.
5. Powtarzaj kroki 1–4, aż wszystkie przykłady pozytywne zostaną pokryte lub zostanie osiągnięty limit reguł.

Przeszukiwanie wiązką (ang. *beam search*)

Ponieważ przestrzeń możliwych kompleksów jest wykładniczo duża, w kroku generacji gwiazdy stosuje się **przeszukiwanie wiązką** o szerokości b . W każdym kroku specjalizacji utrzymywany jest zbiór co najwyżej b najlepiej ocenionych kandydatów, a pozostałe są odrzucane. Szerokość wiązki kontroluje kompromis między jakością rozwiązania a kosztem obliczeniowym.

2.2 Zmodyfikowana wersja ze zbiorem walidacyjnym

W klasycznej wersji algorytmu ocena kandydatów odbywa się na **tym samym zbiorze treningowym**, na którym kompleksy zostały zbudowane. Może to prowadzić do sytuacji analogicznej do nadmiernego dopasowania (*overfitting*): wybierane są kompleksy ściśle dopasowane do danych treningowych, które niekoniecznie uogólniają się na nowe dane.

Proponowana modyfikacja polega na rozdzieleniu dwóch ról zbioru danych:

- **Zbiór treningowy** – służy do generowania i specjalizowania kompleksów (wykluczania przykładów negatywnych). Struktura kompleksów jest zatem wyznaczana na podstawie danych treningowych.
- **Zbiór walidacyjny** – służy wyłącznie do oceny i rankingu kandydatów w każdym kroku przeszukiwania wiązką. Najlepszy kompleks w danej iteracji jest wybierany na podstawie jego F_1 obliczonego na zbiorze walidacyjnym.

Mechanizm ten jest koncepcyjnie zbliżony do techniki wczesnego zatrzymania (*early stopping*) [4] stosowanej przy trenowaniu sieci neuronowych – zewnętrzny sygnał jakości zastępuje ocenę opartą wyłącznie na danych uczących.

Kluczowy fragment implementacji

Metoda `_score_set_for_class` decyduje, który zbiór zostanie użyty do oceny kandydatów w danej iteracji – zwraca "train" dla wariantu klasycznego lub "validation" dla wariantu zmodyfikowanego (z zabezpieczeniem na wypadek braku przykładów walidacyjnych dla danej klasy):

```
def _score_set_for_class(self, target_class):
    if self.variant != "validation":
        return "train"          # wariant klasyczny
    if self._validation_label_masks[target_class].bit_count() == 0:
        return "train"          # fallback: brak przykładów
                                # walidacyjnych dla klasy
    return "validation"         # wariant zmodyfikowany
```

Następnie metoda `_score_candidate` na podstawie zwróconej wartości wybiera odpowiednie maski bitowe i oblicza TP , FP oraz F_1 na właściwym zbiorze:

```
if score_set == "validation":
    full_mask      = self._full_validation_mask
    condition_masks = self._validation_condition_masks
    label_mask     = self._validation_label_masks[target_class]
else:
    full_mask      = self._full_train_mask
    condition_masks = self._train_condition_masks
    label_mask     = self._train_label_masks[target_class]
```

Zastosowanie masek bitowych zamiast iteracji po przykładach znacząco przyspiesza obliczenia dla dużych zbiorów danych (np. Nursery z ponad 12 000 przykładami).

2.3 Metryki oceny

Do porównania wariantów użyto następujących miar:

- **Accuracy** – odsetek poprawnie sklasyfikowanych przykładów.
- **Precision, Recall, F1 (ważone)** – miary ważone wsparciem klas (ang. *weighted*): każda klasa jest uwzględniana proporcjonalnie do liczby jej przykładów; miarodajne przy nierównomiernym rozkładzie klas. W tabelach oznaczane skrótem *Prec.*, *Rec.*, *F1_w*.
- **Macro F1** – średnia arytmetyczna F_1 po klasach bez ważenia; szczególnie czuła na klasy mniejszościowe.
- **Luka train–test** – różnica między wynikiem na zbiorze treningowym a testowym; bezpośrednia miara stopnia przeuczenia. W tabelach oznaczana jako *Luka train–test*.
- **Liczba reguł i średnia złożoność** – miary zwięzłości i interpretowalności modelu.

Wszystkie wyniki są uśrednione po 5 niezależnych losowych podziałach danych (ziarna 0–4). Podawane są wartości średnie wraz z odchyleniem standardowym populacyjnym ($\mu \pm \sigma$).

3 Implementacja

Projekt zaimplementowano w języku **Python 3** bez użycia zewnętrznych bibliotek uczenia maszynowego (takich jak scikit-learn czy pandas). Cały kod indukcji reguł, podziału danych i ewaluacji napisano od podstaw, korzystając wyłącznie z biblioteki standardowej.

Implementacja składa się z dwóch plików:

- **aq_experiments.py** – zawiera klasę `AQClassifier` implementującą oba warianty algorytmu AQ, klasy pomocnicze (`Rule`, `CandidateStats`), stratyfikowany podział danych (`stratified_split`) oraz funkcję ewaluacji (`evaluate_predictions`).
- **evaluate.py** – skrypt uruchamiający eksperymenty porównawcze na wskazanych zbiorach danych, zbierający wyniki per-seed i zapisujący je do pliku CSV.

Do generowania losowych podziałów danych wykorzystano generator pseudolosowy z biblioteki standardowej Pythona (`random.Random`), inicjalizowany kolejnymi ziarenami (0–4), co zapewnia pełną reprodukowalność wyników.

Kluczowe decyzje implementacyjne:

- **Maski bitowe** – każdy przykład jest reprezentowany jako bit w liczbie całkowitej. Operacje AND na maskach zastępują pętle po przykładach, co daje wielokrotne przyspieszenie dla dużych zbiorów.
- **Pamięć podręczna wyników** (*score cache*) – wyniki `_score_candidate` są zapamiętywane w słowniku indeksowanym krotką (`complex`, `target_class`, `score_set`), eliminując wielokrotne obliczenia dla tych samych kandydatów.

- **Wariant algorytmu** – parametr `variant` klasy `AQClassifier` przyjmuje wartość "classic" lub "validation", co pozwala uruchamiać oba warianty tym samym kodem.
- **Klasyfikacja** – przy predykcji stosuje się zasadę pierwszej pasującej reguły z rankingiem według score, precyzji, pokrycia i złożoności. Jeśli żadna reguła nie pasuje, zwracana jest klasa dominująca ze zbioru treningowego.

4 Zbiory danych

W projekcie wykorzystano trzy zbiory danych z repozytorium UCI Machine Learning Repository [3], różniące się rozmiarem, liczbą atrybutów i trudnością klasyfikacji.

4.1 Car Evaluation

- **Liczba przykładów:** 1728
- **Liczba atrybutów:** 6 (wszystkie kategoryczne): `buying`, `maint`, `doors`, `persons`, `lug_boot`, `safety`
- **Klasy:** `unacc` (1210), `acc` (384), `good` (69), `vgood` (65)

Zbiór posiada silnie nierównomierny rozkład klas – klasa `unacc` stanowi około 70% wszystkich przykładów. Jest to naturalny benchmark dla algorytmów indukcji reguł ze względu na czysto symboliczny charakter danych.

4.2 Mushrooms

- **Liczba przykładów:** 8124
- **Liczba atrybutów:** 22 (wszystkie kategoryczne)
- **Klasy:** `p` – trujący (3916), `e` – jadalny (4208)

Zbiór opisuje cechy morfologiczne grzybów. Klasy są dobrze zbalansowane. Dane zawierają silne korelacje atrybutów z klasą (m.in. zapach, kolor wysypu zarodników), co sprawia, że problem jest potencjalnie łatwy do pokrycia zwartymi regułami logicznymi.

4.3 Nursery

- **Liczba przykładów:** 12960
- **Liczba atrybutów:** 8 (wszystkie kategoryczne): `parents`, `has_nurs`, `form`, `children`, `housing`, `finance`, `social`, `health`
- **Klasy:** `not_recom` (4320), `priority` (4266), `spec_prior` (4044), `very_recom` (328), `recommend` (2)

Największy ze zbiorów, zawierający dwie skrajnie mniejszościowe klasy (**recommend**: 2 przykłady, **very_recom**: 328 przykładów). Dane pochodzą z systemu wspomagania decyzji przyjęć do żłobków. Ze względu na rozmiar zbioru czas jednego uruchomienia przy szerokości wiązki $b = 30$ był zbyt długi, aby przeprowadzić pełną serię 5 podziałów w rozsądnym czasie – eksperymenty dotyczące wpływu szerokości wiązki przeprowadzono dla Nursery przy $b = 10$ (zob. sekcja 5.2).

4.4 Podział danych

We wszystkich eksperymentach zastosowano **stratyfikowany podział** danych, gwarantujący zachowanie proporcji klas w każdym ze zbiorów. Domyślny podział:

$$\text{treningowy} = 60\%, \quad \text{walidacyjny} = 20\%, \quad \text{testowy} = 20\%$$

5 Eksperymenty i wyniki

We wszystkich tabelach kolumny *Prec.*, *Rec.* i *F1_W* oznaczają miary ważone wsparciem klas (zob. sekcja 2.3). We wszystkich eksperymentach użyto limitu 200 reguł na klasę (`--max-rules-per-class 200`).

5.1 Eksperyment 1 – Porównanie wariantów (wyniki główne)

Parametry: szerokość wiązki $b = 30$, podział 60/20/20%, limit reguł 200, 5 podziałów losowych (ziarna 0–4).

Car Evaluation

Tabela 1: Car Evaluation – jakość na zbiorze testowym ($\mu \pm \sigma$, $n = 5$)

Wariant	Accuracy	Prec.	Rec.	Macro F1
Klasyczny	0.8740 ± 0.0108	0.8691 ± 0.0105	0.8740 ± 0.0108	0.6827 ± 0.0236
Walidacyjny	0.8699 ± 0.0208	0.8623 ± 0.0251	0.8699 ± 0.0208	0.6491 ± 0.0642

Tabela 2: Car Evaluation – przeuczenie i złożoność modelu ($\mu \pm \sigma$, $n = 5$)

Wariant	Train acc	Luka train–test	Liczba reguł	Śr. warunki
Klasyczny	1.0000 ± 0.0000	0.1260 ± 0.0108	149.8 ± 6.7	4.73 ± 0.04
Walidacyjny	1.0000 ± 0.0000	0.1301 ± 0.0208	169.4 ± 8.5	4.87 ± 0.04

Na zbiorze Car Evaluation oba warianty osiągają zbliżoną dokładność testową (0.8740 vs 0.8699). Wariant klasyczny uzyskuje wyższe Macro F1 (0.6827 vs 0.6491), co wskazuje na lepsze pokrycie klas mniejszościowych. Wariant walidacyjny generuje o 13% więcej reguł (169 vs 150) przy nieco wyższej złożoności warunkowej. Wariancja wyników wariantu walidacyjnego jest niemal dwukrotnie większa, co świadczy o większej wrażliwości na konkretny podział danych.

Mushrooms

Tabela 3: Mushrooms – jakość na zbiorze testowym ($\mu \pm \sigma$, $n = 5$)

Wariant	Accuracy	Prec.	Rec.	Macro F1
Klasyczny	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
Walidacyjny	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000

Tabela 4: Mushrooms – przeuczenie i złożoność modelu ($\mu \pm \sigma$, $n = 5$)

Wariant	Train acc	Luka train–test	Liczba reguł	Śr. warunki
Klasyczny	1.0000 ± 0.0000	0.0000 ± 0.0000	25.8 ± 1.2	3.05 ± 0.01
Walidacyjny	1.0000 ± 0.0000	0.0000 ± 0.0000	26.4 ± 2.1	3.01 ± 0.02

Na zbiorze Mushrooms oba warianty osiągają doskonałą klasyfikację (accuracy = 1.0000) przy zerowej luce train–test i minimalnej liczbie reguł (ok. 26). Zbiór ten zawiera silne, deterministyczne zależności, które algorytm AQ odkrywa niezależnie od wariantu oceny kompleksów.

Nursery

Tabela 5: Nursery – jakość na zbiorze testowym ($b = 30$, $\mu \pm \sigma$, $n = 5$)

Wariant	Accuracy	Prec.	Rec.	Macro F1
Klasyczny	0.9633 ± 0.0023	0.9647 ± 0.0025	0.9633 ± 0.0023	0.7947 ± 0.0798
Walidacyjny	0.9649 ± 0.0039	0.9662 ± 0.0032	0.9649 ± 0.0039	0.8353 ± 0.0864

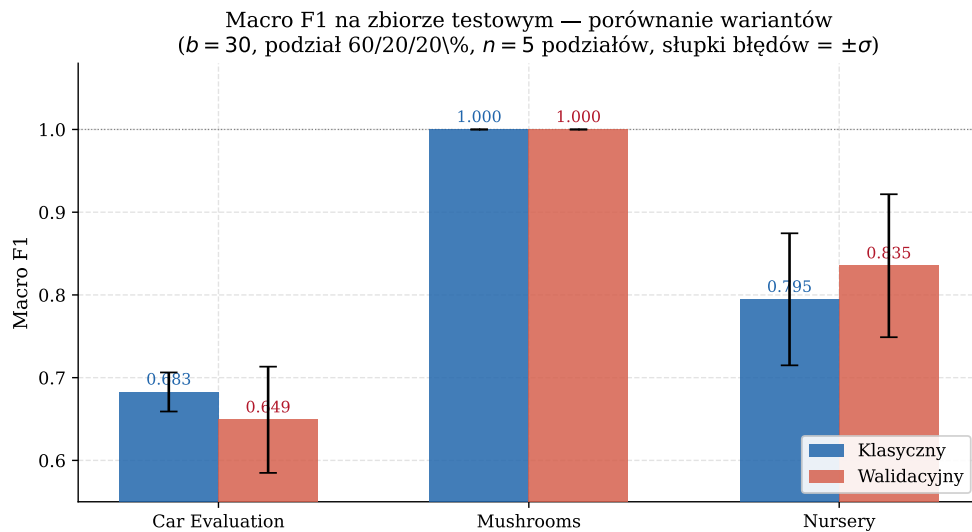
Tabela 6: Nursery – przeuczenie i złożoność modelu ($b = 30$, $\mu \pm \sigma$, $n = 5$)

Wariant	Train acc	Luka train–test	Liczba reguł	Śr. warunki
Klasyczny	0.9974 ± 0.0008	0.0342 ± 0.0025	461.8 ± 3.9	5.14 ± 0.02
Walidacyjny	0.9954 ± 0.0004	0.0305 ± 0.0042	481.0 ± 3.9	5.22 ± 0.02

Nursery jest jedynym zbiorem, na którym wariant walidacyjny osiąga mierzalnie lepsze wyniki testowe: accuracy 0.9649 vs 0.9633 oraz Macro F1 0.8353 vs 0.7947 (poprawa o +0.040). Jednocześnie luka train–test jest mniejsza (0.0305 vs 0.0342), co potwierdza ograniczenie przeuczenia. Obserwacja ta jest szczególnie istotna, gdyż Nursery zawiera skrajnie mniejszościowe klasy – zbiór walidacyjny pozwala lepiej oceniać kompleksy pod kątem ich uogólnienia na rzadkich klasach.

Duże odchylenie standardowe Macro F1 dla obu wariantów ($\sigma \approx 0.08$, widoczne na rysunku 1) jest zjawiskiem oczekiwanym i wynika z dwóch czynników. Po pierwsze, przy zaledwie $n = 5$ podziałach losowych każdy pojedynczy podział ma duży wpływ na wartość średnią. Po drugie, klasa **recommend** liczy tylko 2 przykłady w całym zbiorze – w zależności

od podziału trafia ona w całości do zbioru treningowego albo testowego, co powoduje skokowe zmiany Macro F1 między podziałami. Szerokość słupków błędów odzwierciedla zatem niestabilność miary, a nie niestabilność algorytmu.



Rysunek 1: Macro F1 na zbiorze testowym dla obu wariantów algorytmu na trzech zbiorach danych ($b = 30$, podział 60/20/20%, $n = 5$; słupki błędów oznaczają $\pm \sigma$).

5.2 Eksperyment 2 – Wpływ szerokości wiązki

Badano wpływ parametru $b \in \{10, 30, 60\}$ dla zbiorów Car Evaluation i Mushrooms przy podziale 60/20/20%. Na zbiorze Nursery ze względu na czas obliczeń przy $b = 30$ użyto $b = 10$ jako jedynej wartości bazowej (tabela 8).

Tabela 7: Wpływ szerokości wiązki – Car Evaluation ($\mu \pm \sigma$, $n = 5$)

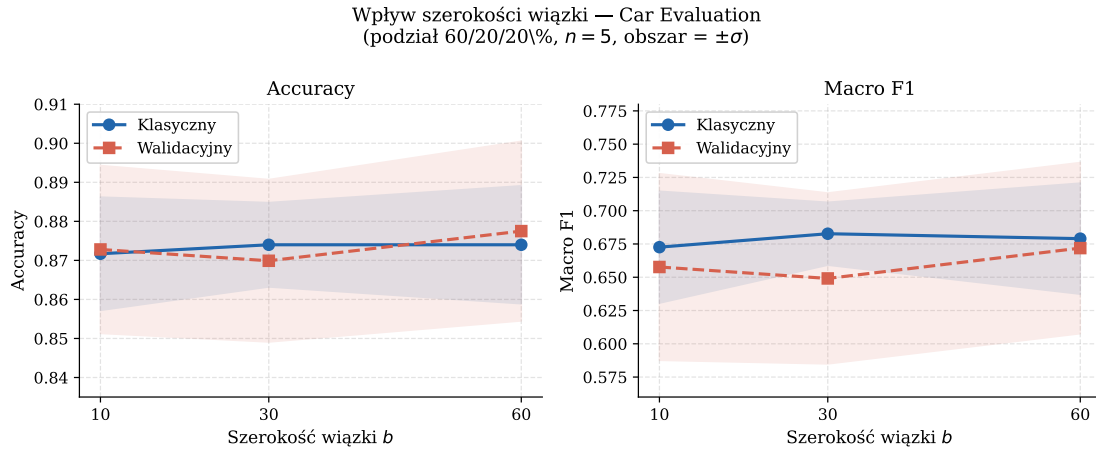
b	Wariant	Accuracy	Macro F1	Luka train-test	Liczba reguł
10	Klasyczny	0.8717 ± 0.0145	0.6726 ± 0.0420	0.1283 ± 0.0145	150.2 ± 8.7
	Walidacyjny	0.8728 ± 0.0215	0.6577 ± 0.0701	0.1272 ± 0.0215	170.2 ± 9.9
30	Klasyczny	0.8740 ± 0.0108	0.6827 ± 0.0236	0.1260 ± 0.0108	149.8 ± 6.7
	Walidacyjny	0.8699 ± 0.0208	0.6491 ± 0.0642	0.1301 ± 0.0208	169.4 ± 8.5
60	Klasyczny	0.8740 ± 0.0151	0.6790 ± 0.0417	0.1260 ± 0.0151	150.4 ± 6.4
	Walidacyjny	0.8775 ± 0.0230	0.6719 ± 0.0643	0.1225 ± 0.0230	168.4 ± 10.4

Tabela 8: Nursery – oba warianty przy $b = 10$ ($\mu \pm \sigma$, $n = 5$)

Wariant	Accuracy	Macro F1	Luka train-test	Liczba reguł
Klasyczny	0.9588 ± 0.0038	0.7579 ± 0.0695	0.0354 ± 0.0043	465.8 ± 3.1
Walidacyjny	0.9601 ± 0.0038	0.8343 ± 0.0874	0.0308 ± 0.0038	481.2 ± 5.2

Szerokość wiązki nie ma istotnego wpływu na accuracy testową – różnice między $b = 10$, $b = 30$ i $b = 60$ mieszczą się w granicach odchylenia standardowego dla obu wariantów

na zbiorze Car Evaluation. Wariant walidacyjny generuje konsekwentnie o ok. 12–13% więcej reguł niż klasyczny niezależnie od wartości b . Przy $b = 60$ wariant walidacyjny uzyskuje nieznacznie wyższą accuracy (0.8775 vs 0.8740) i niższą lukę train–test (0.1225 vs 0.1260), co sugeruje, że szersza wiązka daje większy wybór kandydatów i pozwala zbiorowi walidacyjnemu wybrać bardziej ogólne kompleksy. Na zbiorze Nursery przy $b = 10$ wzorec jest taki sam jak przy $b = 30$: wariant walidacyjny osiąga wyższe Macro F1 (0.8343 vs 0.7579) i mniejszą lukę train–test (0.0308 vs 0.0354).



Rysunek 2: Wpływ szerokości wiązki b na accuracy i Macro F1 - Car Evaluation (podział 60/20/20%, $n = 5$; obszar zacieniowany $= \pm\sigma$).

5.3 Eksperyment 3 – Wpływ rozmiaru zbioru walidacyjnego

Badano wpływ proporcji zbioru walidacyjnego przy $b = 30$ i limicie reguł 200. Rozmiar zbioru testowego pozostał stały (20%); zmieniano proporcje między zbiorem treningowym a walidacyjnym.

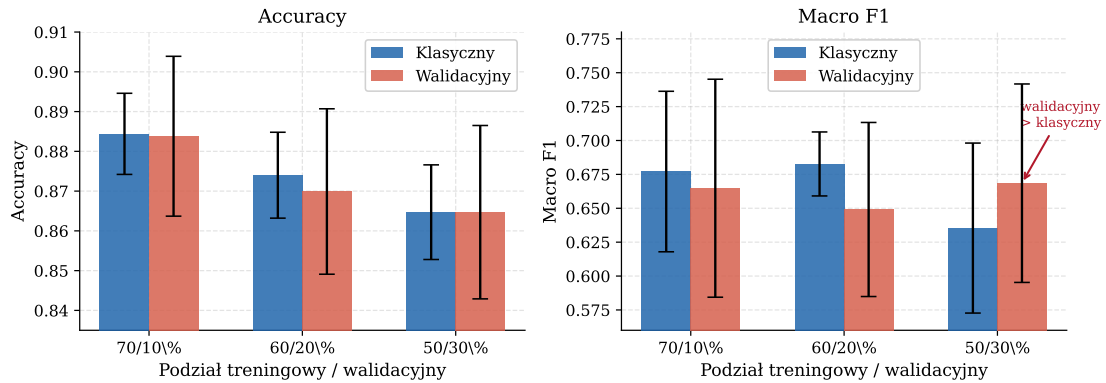
Tabela 9: Wpływ rozmiaru zbioru walidacyjnego – Car Evaluation ($\mu \pm \sigma$, $n = 5$)

Podział tr/val	Wariant	Accuracy	Macro F1	Liczba reguł
70% / 10%	Klasyczny	0.8844 ± 0.0102	0.6771 ± 0.0592	170.2 ± 11.6
	Walidacyjny	0.8838 ± 0.0201	0.6648 ± 0.0804	190.0 ± 11.4
60% / 20%	Klasyczny	0.8740 ± 0.0108	0.6827 ± 0.0236	149.8 ± 6.7
	Walidacyjny	0.8699 ± 0.0208	0.6491 ± 0.0642	169.4 ± 8.5
50% / 30%	Klasyczny	0.8647 ± 0.0119	0.6354 ± 0.0627	130.4 ± 6.0
	Walidacyjny	0.8647 ± 0.0218	0.6685 ± 0.0732	147.6 ± 7.8

Zmniejszenie zbioru treningowego (na rzecz większego walidacyjnego) obniża accuracy obu wariantów – mniejszy zbiór treningowy ogranicza liczbę wygenerowanych reguł (130 vs 150 vs 170), zmniejszając pokrycie przestrzeni cech.

Interesującą obserwacją jest zachowanie Macro F1: przy podziale 50/30% wariant walidacyjny uzyskuje wyższe Macro F1 niż klasyczny (0.6685 vs 0.6354), podczas gdy przy podziale 70/10% sytuacja jest odwrotna (0.6648 vs 0.6771). Wskazuje to, że większy zbiór walidacyjny poprawia ocenę rzadkich klas, bez istotnego wpływu na accuracy.

Wpływ rozmiaru zbioru walidacyjnego — Car Evaluation
($b = 30$, zbiór testowy stały = 20%, $n = 5$, słupki błędów = $\pm\sigma$)



Rysunek 3: Wpływ rozmiaru zbioru walidacyjnego na accuracy i Macro F1 - Car Evaluation ($b = 30$, zbiór testowy stały = 20%, $n = 5$; strzałka wskazuje konfigurację, w której wariant walidacyjny przewyższa klasyczny pod względem Macro F1).

6 Wnioski i podsumowanie

Tabela 10: Zestawienie wyników głównego eksperymentu ($b = 30$, podział 60/20/20%)

Zbiór	Wariant	Accuracy	Macro F1	Luka train-test
Car Evaluation	Klasyczny	0.8740 ± 0.0108	0.6827 ± 0.0236	0.1260 ± 0.0108
	Walidacyjny	0.8699 ± 0.0208	0.6491 ± 0.0642	0.1301 ± 0.0208
Mushrooms	Klasyczny	1.0000 ± 0.0000	1.0000 ± 0.0000	0.0000 ± 0.0000
	Walidacyjny	1.0000 ± 0.0000	1.0000 ± 0.0000	0.0000 ± 0.0000
Nursery	Klasyczny	0.9633 ± 0.0023	0.7947 ± 0.0798	0.0342 ± 0.0025
	Walidacyjny	0.9649 ± 0.0039	0.8353 ± 0.0864	0.0305 ± 0.0042

Na podstawie przeprowadzonych eksperymentów sformułowano następujące wnioski:

1. **Wpływ modyfikacji zależy od charakterystyki zbioru danych.** Wariant walidacyjny przynosi wyraźne korzyści na zbiorze Nursery – dużym, wieloklasowym, z silnie nierównomiernym rozkładem: accuracy +0.0016, Macro F1 +0.0406, mniejsza luka train-test (0.0305 vs 0.0342). Na zbiorze Car Evaluation różnice są marginalne i niejednoznaczne, natomiast na zbiorze Mushrooms, gdzie dane są niemal liniowo separowalne regułami logicznymi, oba warianty dają identyczne wyniki (accuracy = 1.0). Modyfikacja ma zatem potencjał przede wszystkim w scenariuszach z wieloma klasami, szumem lub nakładającymi się obszarami decyzyjnymi.
2. **Wariant walidacyjny generuje więcej reguł.** We wszystkich konfiguracjach wariant walidacyjny tworzy konsekwentnie więcej reguł niż klasyczny: ok. 12–13% dla Car Evaluation i ok. 3–4% dla Nursery. Wynika to z faktu, że kompleksy wybrane na podstawie zbioru walidacyjnego są zazwyczaj bardziej zachowawcze (wyższa precyzja, mniejsze pokrycie), przez co w każdej iteracji usuwają mniej przykładów z puli niepokrytych, wymagając większej liczby reguł do pokrycia całości.

3. **Wariant walidacyjny cechuje się większą wariancją wyników.** Odchylenie standardowe wyników wariantu walidacyjnego jest konsekwentnie wyższe niż klasycznego. Jest to koszt uzależnienia rankingu kandydatów od stosunkowo małego zbioru walidacyjnego, który może nie być reprezentatywny dla wszystkich klas przy każdym podziale.
4. **Większy zbiór walidacyjny sprzyja klasom mniejszościowym.** Przy podziale 50/30% wariant walidacyjny osiąga wyższe Macro F1 niż klasyczny (0.6685 vs 0.6354), przy identycznej accuracy (0.8647). Większy zbiór walidacyjny lepiej reprezentuje rzadkie klasy i pozwala na korzystniejszy wybór kompleksów dla klas mniejszościowych.
5. **Szerokość wiązki ma marginalne znaczenie.** W zakresie $b \in \{10, 30, 60\}$ nie zaobserwowano istotnych różnic w accuracy. Większa wiązka nieznacznie sprzyja wariantowi walidacyjnemu (niższa luka train-test przy $b = 60$), lecz efekt nie jest duży w stosunku do odchylenia standardowego wyników.

Ograniczenia metodologiczne

Przeprowadzone eksperymenty mają kilka istotnych ograniczeń, które należy wziąć pod uwagę przy interpretacji wyników:

- **Mała liczba podziałów ($n = 5$).** Uśrednianie po zaledwie 5 losowych podziałach danych daje stosunkowo szerokie przedziały ufności. Różnice między wariantami na poziomie $|\Delta_{\text{acc}}| < 0.005$ (Car Evaluation) mogą nie być istotne statystycznie – weryfikacja wymagałaby testu, np. Wilcozona dla prób sparowanych lub co najmniej $n = 30$ podziałów.
- **Brak testów istotności statystycznej.** Wyniki prezentowane są wyłącznie jako $\mu \pm \sigma$; nie przeprowadzono formalnych testów hipotez. Sformułowane wnioski mają charakter eksploracyjny.
- **Ograniczony zakres hiperparametrów.** Przeszukiwano jedynie trzy wartości szerokości wiązki i trzy proporcje podziału danych; inne kombinacje mogłyby ujawnić inne zależności.
- **Jeden typ funkcji oceny.** Zarówno podczas budowy gwiazdy, jak i przy ocenie na zbiorze walidacyjnym używano miary F_1 . Użycie innych miar (np. samej precyzji lub pokrycia) mogłoby inaczej ukształtować zachowanie wariantu walidacyjnego.

Podsumowanie. Zmodyfikowana wersja algorytmu AQ z oceną kompleksów na zbiorze walidacyjnym jest poprawnie zaimplementowana i działa zgodnie z założeniami projektu. Wyniki eksperymentów wskazują, że modyfikacja przynosi mierzalne korzyści przede wszystkim na zbiorach dużych i wieloklasowych z nierównomiernym rozkładem klas (Nursery), gdzie ogranicza przeuczenie i poprawia Macro F1. Na zbiorach z silnymi deterministycznymi zależnościami lub mniejszych różnice między wariantami są marginalne. Kosztem modyfikacji jest większa liczba generowanych reguł i wyższa wrażliwość na podział danych, co przy małej liczbie podziałów utrudnia jednoznaczne wnioskowanie statystyczne.

Literatura

- [1] Michalski, R.S. (1969). *On the quasi-minimal solution of the general covering problem*. Proceedings of the 5th International Symposium on Information Processing (FCIP 69), Bled, Yugoslavia, vol. A3, pp. 125–128.
- [2] Michalski, R.S. (1983). A theory and methodology of inductive learning. W: Michalski, R.S., Carbonell, J.G., Mitchell, T.M. (red.), *Machine Learning: An Artificial Intelligence Approach*, vol. 1, pp. 83–134. Tioga Publishing, Palo Alto, CA.
- [3] Dua, D., Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences. Dostępne pod adresem: <https://archive.ics.uci.edu>
- [4] Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer, New York, NY.